

MANUAL TÉCNICO

Integração CI/CD

Sistema de cadastro de clientes em JavaScript

Source → Testes unitários → GitHub Actions → Docker Hub → Render

IDENTIFICAÇÃO	INFORMAÇÃO
Autor	João Emanuel de Souza da Silva
Projeto	CadastroFlow
Tecnologias	JavaScript, Node.js, Docker, Nginx, GitHub Actions, Docker Hub e Render
Data	2026

Sumário

- 1** Apresentação do projeto
- 2** Estrutura do repositório Git
- 3** Visão geral do pipeline
- 4** Infraestrutura utilizada
- 5** Dockerfile e build da imagem
- 6** Execução dos testes unitários
- 7** Workflow do GitHub Actions
- 8** Secrets e credenciais
- 9** Publicação no Docker Hub
- 10** Deploy automático no Render
- 11** Primeira publicação no GitHub
- 12** Dificuldades e soluções
- 13** Critérios de aceite
- 14** Conclusão
- A** Workflow completo
- B** Comandos rápidos

1. Apresentação do projeto

Este manual descreve o desenvolvimento e, principalmente, a entrega automática do CadastroFlow: um sistema web de cadastro de clientes feito com HTML, CSS e JavaScript. O objetivo do pipeline é transformar cada alteração enviada à branch principal em uma versão testada, empacotada em Docker e publicada na nuvem sem copiar arquivos manualmente para o servidor.

Resultado esperado Depois de um push na branch main, o GitHub Actions executa oito testes. Se todos passarem, cria e envia uma imagem ao Docker Hub e chama o Deploy Hook do Render. Se qualquer teste falhar, a publicação e o deploy não são executados.

1.1 Funcionalidade principal

- Cadastro de cliente com nome, e-mail, telefone e cidade.
- Validação de campos obrigatórios, formato de e-mail, telefone e e-mail duplicado.
- Edição e exclusão de registros.
- Busca por nome, e-mail ou cidade.
- Persistência local no navegador por meio de localStorage.

1.2 Entrada, processamento e saída

ETAPA	NO SISTEMA
Entrada	Dados preenchidos no formulário pelo usuário.
Processamento	Validação, normalização do e-mail, verificação de duplicidade e gravação.
Saída	Mensagem de sucesso ou erro e lista de clientes atualizada na tela.

1.3 Limite da solução

Por ser um projeto didático front-end, os registros são salvos somente no localStorage do navegador. O foco do trabalho é demonstrar CI/CD. Em uma evolução real, a camada de armazenamento pode ser substituída por uma API e banco de dados sem alterar a lógica principal do pipeline.

2. Estrutura do repositório Git

O GitHub é a origem oficial do código. O arquivo que inicia a automação está em `.github/workflows/ci-cd.yml`. O evento push na branch main dispara o pipeline. Pull requests para main executam somente os testes, protegendo a branch antes da integração.

Estrutura entregue no arquivo ZIP

```
cadastro-clientes-cicd/
├── .github/workflows/ci-cd.yml # pipeline
├── css/styles.css             # aparência
├── js/app.js                  # interface e eventos
├── js/customerService.js      # regras testáveis
├── tests/customerService.test.js # 8 testes
├── Dockerfile                 # imagem de produção
├── nginx.conf                 # servidor web
├── docker-compose.yml         # execução local
├── index.html                  # tela principal
├── package.json                # comando npm test
└── README.md                  # instruções rápidas
```

2.1 Separação da lógica

As regras de negócio ficam em `js/customerService.js` e não dependem da tela. Essa separação permite testar validação, criação, busca e armazenamento diretamente com o Node.js. O arquivo `js/app.js` cuida do DOM, eventos do formulário e renderização da lista.

2.2 O que deve ser enviado ao GitHub

Todos os arquivos do ZIP devem entrar no repositório, exceto itens ignorados por `.gitignore`. O `Dockerfile` fica na raiz porque o workflow usa `context: .` durante o build. A pasta `.github` deve permanecer com o ponto inicial; se ela for renomeada, o GitHub não encontrará o workflow.

3. Visão geral do pipeline

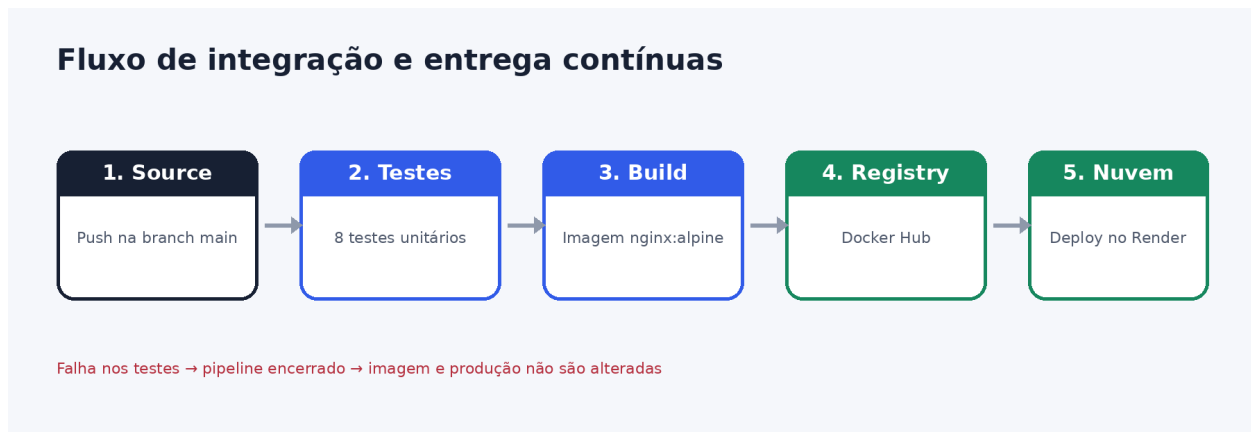


Figura 1 — Pipeline implementado no projeto.

3.1 Fluxo automático

1. O desenvolvedor altera o sistema, cria um commit e executa git push origin main.
2. O GitHub reconhece o gatilho push e inicia o job testes em uma máquina Ubuntu temporária.
3. O Node.js executa npm test. Os oito testes precisam terminar com sucesso.
4. O job publicar-imagem autentica no Docker Hub, constrói a imagem e publica as tags latest e o SHA do commit.
5. O job deploy faz uma requisição POST ao Deploy Hook do Render.
6. O Render baixa a imagem atualizada do Docker Hub e substitui o container em produção.

3.2 Dependência entre os jobs

A diretiva needs cria uma sequência obrigatória: publicar-imagem depende de testes; deploy depende de publicar-imagem. Portanto, falhas são propagadas. Um job dependente fica marcado como skipped quando o job anterior falha.

Comportamento em caso de teste com falha

```

testes → publicar-imagem → deploy
×      skipped      skipped
  
```

4. Infraestrutura utilizada

COMPONENTE	RESPONSABILIDADE	LOCAL
Git/GitHub	Versionar o código e registrar cada alteração.	Repositório remoto
Node.js 22	Executar testes unitários com node:test.	Runner do GitHub Actions
Docker	Montar uma imagem reproduzível do site.	Runner e computador local
nginx:alpine	Servir os arquivos estáticos na porta 80.	Dentro do container
Docker Hub	Armazenar as imagens latest e SHA.	Registro de containers
Render	Executar a imagem e disponibilizar a URL pública.	Nuvem

4.1 Por que nginx:alpine

A imagem nginx:alpine foi escolhida por ser apropriada para arquivos HTML, CSS e JavaScript estáticos. Ela combina o servidor Nginx com uma base Alpine pequena. Não é necessário manter Node.js dentro do container de produção, porque o projeto não possui compilação front-end nem servidor Node.

4.2 Responsabilidades do ambiente

- Computador do desenvolvedor: escrever e testar o código, opcionalmente executar Docker Compose.
- GitHub Actions: validar, construir e publicar automaticamente em ambiente limpo.
- Docker Hub: manter a imagem pronta para distribuição.
- Render: puxar a imagem e mantê-la executando como serviço web.

5. Dockerfile e build da imagem

Dockerfile completo do projeto

```
FROM nginx:alpine

COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY index.html /usr/share/nginx/html/index.html
COPY css /usr/share/nginx/html/css
COPY js /usr/share/nginx/html/js

EXPOSE 80
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD wget --quiet --tries=1 --spider http://localhost/ || exit 1
```

5.1 Explicação linha a linha

INSTRUÇÃO	FUNÇÃO
FROM nginx:alpine	Define a imagem-base com Nginx.
COPY nginx.conf ...	Substitui a configuração padrão do servidor.
COPY index.html/css/js ...	Copia somente os arquivos necessários para produção.
EXPOSE 80	Documenta a porta HTTP usada pelo container.
HEALTHCHECK ...	Verifica periodicamente se o site responde.

5.2 Build e execução local

```
docker compose up -d --build
# abrir: http://localhost:8080

docker compose ps
docker compose logs -f
docker compose down
```

O parâmetro `--build` força a recriação da imagem. A porta 8080 do computador é encaminhada para a porta 80 do container. O comando deve ser executado dentro da pasta que contém `docker-compose.yml`; caso contrário, aparece o erro “no configuration file provided”.

5.3 Conteúdo excluído

O arquivo `.dockerignore` impede o envio de testes, histórico Git, documentos e dependências locais para o contexto de build. Isso reduz o tamanho transferido e evita incluir arquivos que não pertencem à produção.

6. Execução dos testes unitários

Os testes usam o módulo nativo `node:test` e `assert/strict`. Não há dependências externas; por isso o pipeline configura o Node.js e executa `npm test` diretamente. O comando definido em `package.json` é `node --test tests/*.test.js`.

6.1 Casos cobertos

TESTE	VERIFICAÇÃO
01	Aceita cliente válido
02	Rejeita nome curto, e-mail e telefone inválidos
03	Bloqueia e-mail duplicado
04	Permite o próprio e-mail durante edição
05	Normaliza nome e e-mail
06	Busca por nome, e-mail ou cidade
07	Salva e carrega do armazenamento
08	Trata armazenamento corrompido

6.2 Posição no pipeline

Os testes são o primeiro job. O workflow só libera a construção e publicação da imagem quando o job testes fica verde. Isso evita que uma versão com regra de cadastro quebrada chegue ao Docker Hub ou ao Render.

Comando executado localmente e no GitHub Actions

```
npm test

# resultado esperado
# tests 8
```



```
# pass 8
# fail 0
```

Regra de bloqueio Se um teste retornar erro, npm termina com código diferente de zero. O GitHub Actions marca o job como failed e não inicia os jobs que possuem needs: testes.

7. Workflow do GitHub Actions

O workflow está em `.github/workflows/ci-cd.yml`. Um workflow é um processo automatizado descrito em YAML e dividido em jobs e steps. O arquivo entregue possui três jobs: testes, publicar-imagem e deploy.

7.1 Gatilhos

```
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:
```

- push na main: testa, publica a imagem e faz deploy.
- pull_request para main: executa somente os testes; a condição if impede publicação.
- workflow_dispatch: permite iniciar o workflow manualmente para diagnóstico.

7.2 Job de testes

```
testes:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: 22
    - run: npm test
```

7.3 Job de publicação

A condição if restringe este job ao evento push na referência refs/heads/main. Depois do login, build-push-action monta a imagem e publica duas tags: latest, para o serviço de produção, e github.sha, para rastrear exatamente qual commit gerou cada versão.

7.4 Job de deploy

Depois que a imagem é publicada, curl envia POST para a URL secreta do Deploy Hook. O parâmetro --fail faz o step retornar erro se o Render responder com status HTTP de falha.

8. Secrets e credenciais

Credenciais nunca devem ser gravadas no YAML. Elas ficam criptografadas na configuração do repositório e são acessadas pela sintaxe secrets.NOME. Para cadastrar: GitHub → repositório → Settings → Secrets and variables → Actions → New repository secret.

SECRET	COMO OBTER	USO
DOCKERHUB_USERNAME	Nome público da conta Docker Hub.	Forma o nome da imagem e realiza login.
DOCKERHUB_TOKEN	Docker Hub → Account settings → Personal access tokens.	Autenticação; usar token em vez da senha.
RENDER_DEPLOY_HOOK_URL	Render → serviço → Settings → Deploy Hook.	Aciona a atualização do serviço.

8.1 Cuidados de segurança

- Não colocar tokens no README, Dockerfile, código ou prints públicos.
- Dar ao token apenas as permissões necessárias para publicar a imagem.
- Revogar e criar outro token se ele for exposto.
- Não imprimir o conteúdo dos secrets com echo durante o pipeline.

Importante O ZIP contém o pipeline pronto, mas não contém credenciais pessoais. O deploy real só começa depois que o proprietário criar as contas e cadastrar os três secrets no GitHub.

9. Publicação no Docker Hub

9.1 Preparar o repositório de imagem

1. Entre em hub.docker.com e crie uma conta.
2. Selecione Create repository.
3. Use o nome cadastro-clientes.
4. Defina a visibilidade pública para a configuração mais simples, ou privada se o plano permitir e o Render receber credencial.
5. Crie um Personal Access Token e salve-o no secret DOCKERHUB_TOKEN.

9.2 Imagens geradas

```
SEU_USUARIO/cadastro-clientes:latest  
SEU_USUARIO/cadastro-clientes:<SHA_DO_COMMIT>
```

latest representa a versão mais recente aprovada. A tag de SHA preserva rastreabilidade e pode ser usada para rollback. Exemplo: um commit 4b28... gera cadastro-clientes:4b28....

9.3 Autenticação automatizada

docker/login-action recebe o nome de usuário e o token pelos secrets. O token é disponibilizado somente durante o job. Em seguida, docker/build-push-action usa push: true para enviar as tags ao Docker Hub.

10. Deploy automático no Render

10.1 Criar o serviço

1. No painel do Render, clique em New + e depois Web Service.
2. Escolha Existing Image.
3. Informe `docker.io/SEU_USUARIO/cadastro-clientes:latest`.
4. Se a imagem for privada, adicione a credencial do Docker Hub solicitada pelo Render.
5. Defina Health Check Path como `/health` e conclua a criação do serviço.
6. Abra Settings, localize Deploy Hook e copie a URL.
7. No GitHub, salve essa URL como `RENDER_DEPLOY_HOOK_URL`.

10.2 Como ocorre a atualização

Serviços criados a partir de imagem externa não acompanham automaticamente cada push no registro. Por isso o último job chama o Deploy Hook. Ao receber a requisição, o Render puxa novamente a tag `latest` e inicia um novo deploy.

10.3 Conferência

- GitHub → Actions: os três jobs devem aparecer em verde.
- Docker Hub → Tags: `latest` e o SHA atual devem estar visíveis.
- Render → Deploys: o deploy mais recente deve indicar sucesso.
- URL do serviço: abrir a tela e cadastrar um cliente de teste.

Observação sobre porta A imagem Nginx escuta na porta 80. O Render encaminha o tráfego público para o serviço; não é necessário expor a porta 8080 em produção.

11. Primeira publicação no GitHub

Depois de criar um repositório vazio no GitHub, execute estes comandos dentro da pasta do projeto. Substitua SEU_USUARIO e SEU_REPOSITORIO.

```
git init
git add .
git commit -m "Projeto inicial com pipeline CI/CD"
git branch -M main
git remote add origin https://github.com/SEU_USUARIO/SEU_REPOSITORIO.git
git push -u origin main
```

O primeiro push executará imediatamente o workflow. Se os secrets ainda não existirem, os testes poderão passar, mas o login no Docker Hub falhará. Recomenda-se cadastrar as credenciais antes do primeiro push ou executar novamente com Run workflow após configurá-las.

11.1 Alteração seguinte

```
git add .
git commit -m "Ajusta validação do cadastro"
git push
```

A partir desse ponto, cada push na main repete o ciclo completo. O histórico do Git permite identificar qual mudança corresponde à tag SHA publicada.

12. Dificuldades encontradas e soluções

DIFICULDADE	CAUSA	SOLUÇÃO
Testar código ligado à tela	Funções misturadas com manipulação do DOM.	Separar regras em <code>customerService.js</code> .
Deploy mesmo com teste quebrado	Jobs independentes poderiam executar em paralelo.	Usar <code>needs</code> para bloquear publicação e deploy.
Expor credenciais	Tokens escritos diretamente no YAML.	Usar GitHub Actions Secrets.
Imagem com arquivos desnecessários	Contexto Docker enviava todo o projeto.	Criar <code>.dockerignore</code> e copiar só <code>index</code> , <code>css</code> e <code>js</code> .
Render não atualiza imagem sozinho	Serviço usa imagem externa do Docker Hub.	Chamar Deploy Hook depois do push da imagem.
Compose não encontrado	Comando executado fora da pasta do YAML.	Abrir o terminal na raiz do projeto.

12.1 Estratégia de diagnóstico

1. Abrir GitHub → Actions e identificar o primeiro step vermelho.
2. Se for teste, executar `npm test` localmente e corrigir a regra.
3. Se for login, conferir nome dos secrets e validade do token.
4. Se for build, executar `docker build -t cadastro-clientes:testes .` localmente.
5. Se for deploy, testar o Deploy Hook e conferir os logs do Render.

13. Critérios de aceite

- ☐ npm test mostra 8 testes aprovados e 0 falhas.
- ☐ docker compose up -d --build disponibiliza o site em localhost:8080.
- ☐ Push na main inicia o workflow automaticamente.
- ☐ Teste propositalmente quebrado impede os jobs seguintes.
- ☐ Docker Hub recebe latest e uma tag com SHA.
- ☐ Render registra novo deploy após a publicação.
- ☐ A URL pública exibe o sistema de cadastro funcional.

13.1 Demonstração sugerida

Para apresentar, mostre o cadastro funcionando; depois abra tests/customerService.test.js e execute npm test. Em seguida, mostre Dockerfile, o arquivo ci-cd.yml e a aba Actions. Finalize exibindo as tags no Docker Hub e a execução no Render. Explique que o teste funciona como portão: sem aprovação, não há nova versão em produção.

14. Conclusão

O projeto demonstra um fluxo completo de CI/CD para uma aplicação JavaScript. O GitHub concentra o código; os testes automatizados validam as regras; Docker e Nginx criam uma unidade reproduzível; Docker Hub distribui a imagem; e Render executa a versão aprovada. A dependência entre os jobs garante que uma falha de qualidade interrompa a entrega antes da produção.

14.1 Referências

GitHub Docs — Workflow syntax. Disponível em:

<https://docs.github.com/actions/using-workflows/workflow-syntax-for-github-actions>

GitHub Docs — Using secrets. Disponível em: <https://docs.github.com/actions/security-guides/using-secrets-in-github-actions>

Docker Docs — Build with GitHub Actions. Disponível em: <https://docs.docker.com/build/ci/github-actions/>

Render Docs — Deploy Hooks. Disponível em: <https://render.com/docs/deploy-hooks>

Render Docs — Deploying an image. Disponível em: <https://render.com/docs/deploying-an-image>

Material indicado na atividade. Disponível em: <https://github.com/marcosnielsen/aula-github-actions>

Acesso às documentações oficiais em setembro de 2026.

Apêndice A — Workflow completo

```

name: CI/CD - Testar, publicar e implantar

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read

jobs:
  testes:
    name: Testes unitários
    runs-on: ubuntu-latest
    steps:
      - name: Baixar código do repositório
        uses: actions/checkout@v4

      - name: Configurar Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 22

      - name: Executar testes
        run: npm test

  publicar-imagem:
    name: Publicar imagem no Docker Hub
    needs: testes
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - name: Baixar código do repositório
        uses: actions/checkout@v4

      - name: Configurar Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Autenticar no Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKERHUB_USERNAME }
          password: ${ secrets.DOCKERHUB_TOKEN }

      - name: Construir e enviar imagem
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: |
            ${ secrets.DOCKERHUB_USERNAME }/cadastro-clientes:latest
            ${ secrets.DOCKERHUB_USERNAME }/cadastro-clientes:${ github.sha }
          cache-from: type=gha
          cache-to: type=gha,mode=max

  deploy:
    name: Deploy automático no Render
    needs: publicar-imagem
    runs-on: ubuntu-latest
    steps:
      - name: Acionar deploy hook do Render
        env:
          RENDER_DEPLOY_HOOK_URL: ${ secrets.RENDER_DEPLOY_HOOK_URL }
        run: curl --fail --show-error --silent --request POST "$RENDER_DEPLOY_HOOK_URL"

```

Apêndice B — Comandos rápidos

OBJETIVO	COMANDO
Rodar testes	npm test
Criar e iniciar container	docker compose up -d --build
Ver containers	docker compose ps
Acompanhar logs	docker compose logs -f
Encerrar	docker compose down
Enviar alteração	git add . && git commit -m "mensagem" && git push